Technical Documentation

ESR Viewer — R&D Evaluation Dashboard

System Engineering & Analytical Documentation

July 30, 2026

Abstract

This document provides complete technical specifications for the **ESR Viewer** web platform. Built using Python, Streamlit, Pandas, Matplotlib, Seaborn, and Plotly, the platform enables interactive quantitative and qualitative analysis of Evaluation Summary Reports (ESR) across temporal, geographical, and categorical dimensions.

1 Overview & Objectives

The primary goal of the **ESR Viewer** application is to transform raw R&D proposal evaluation data into actionable intelligence. Evaluation Summary Reports contain scores across multiple criteria (Blocks) as well as descriptive feedback (Issues/Remarks).

Key operational capabilities include:

- Multi-dimensional filtering across Area, Action/Project Type, and Evaluation Blocks.
- Longitudinal analysis of score progressions over time.
- Benchmarking of subject areas via unified global heatmaps.
- Evaluation volume tracking and call breakdown.
- Qualitative critique density analysis and correlation modeling with numerical scores.

2 System Architecture & Tech Stack

The application follows a reactive, single-page dashboard design powered by Streamlit and standard Python data science libraries:

3 Input Data Specification

The system ingests tabular data from an Excel dataset formatted as long/tidy data (`esr_data_long.xlsx`).

Table 1: System Component Matrix

Layer	Technology	Role in System
UI Framework	Streamlit 1.x	State management, sidebar layout, dynamic multi-tab presentation.
Data Processing	Pandas	Data loading, cleaning, aggregation, dynamic pivoting, and merging.
Static Rendering	Matplotlib & Seaborn	Generation of annotated heatmaps and clean standard line plots.
Interactive Charts	Plotly Express & GO	Multi-axis time series, stacked bars, and grouped issue charts.

Table 2: Data Schema Requirements

Field Name	Data Type	Description
`Year`	String / Numeric	Year of evaluation (cast to string upon import).
`Area`	String	R&D domain or scientific panel category.
`Type`	String	Action scheme or project type (e.g., RIA, IA, CSA).
`Block`	String	Evaluation criterion (e.g., Excellence, Impact).
`Call`	String	Specific funding call identifier.
`Metric`	String	Distinguishes numerical scores (`Score`) from critique types.
`Sum`	Numeric	Total aggregated value or frequency count for the row group.
`Count`	Numeric	Total evaluation instances represented in the row group.

4 Core Software Architecture & Workflow

4.1 Data Ingestion & Caching

To optimize UI responsiveness, data loading is cached using Streamlit's native memory management decorators.

```
file_path = "esr_data_long.xlsx"

@st.cache_data
def load_data(path):
    df = pd.read_excel(path)
    df['Year'] = df['Year'].astype(str)
    return df
```

If the source dataset is missing, the platform catches `FileNotFoundError` gracefully, rendering an error alert and terminating script execution via `st.stop()`.

4.2 Data Partitioning & Filter Cascade

The dataset is split into two functional subsets:

1. **Scores Dataset (`df_scores`)**: Rows where `Metric == 'Score'`.
2. **Issues Dataset (`df_issues`)**: Rows representing qualitative observations (`Metric != 'Score'`).

Sidebar widgets provide reactive controls:

- **Area**: Single selection dropdown with default "All".
- **Type**: Single selection dropdown with default "All".
- **Block**: Multi-select widget initialized to select all available blocks.

5 Visual Analytics Modules (Tabs Breakdown)

The user interface partitions insights into five dedicated analytical perspectives.

5.1 Tab 1: Trend Analysis (Temporal Evolution)

Computes the total sum and count grouped by `Year` and `Block` to render mean score evolution curves:

$$\text{Average Score} = \frac{\sum \text{Sum}}{\sum \text{Count}} \tag{1}$$

The output is rendered as a Seaborn multi-line plot displaying score trajectories across years for selected evaluation criteria.

5.2 Tab 2: Strengths Map (Heatmap Matrix)

Generates a global overview matrix comparing performance across all `Area` domains versus `Block` criteria.

- **Intentional Bypass**: The user's active `Area` filter is bypassed in this view to allow context-aware benchmarking against global averages.
- **Visualization**: Annotated Seaborn heatmap using the `RdYlGn` diverging palette.

5.3 Tab 3: Volume & Call Distribution

Presents project evaluation counts grouped by evaluation year and specific funding calls. Displayed using a Plotly interactive stacked bar chart.

5.4 Tab 4: Non-Score Issue Analyzer

Aggregates non-score issue occurrences across evaluation criteria blocks (`Block`) and observation types (`Metric`), providing insight into critical bottlenecks or common proposal weaknesses.

5.5 Tab 5: Score vs. Critique Frequency Dynamics

Analyzes the potential impact of critique density on score outcomes.

1. **Unified Dual Y-Axis Chart**: Displays overall average score on the primary vertical axis (bold line) alongside normalized mean appearances of various critique keywords on the secondary vertical axis.
2. **Statistical Correlation Heatmap**: Computes Pearson correlation coefficients (r) between mean scores and critique metrics:

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}} \tag{2}$$

Rendered on a bounded scale $[-1.0, 1.0]$ using a `coolwarm` color scheme.

3. **Consolidated Data Table**: Formatted numerical table formatted to 3 decimal places for detailed audit.

6 Deployment & Local Execution

To launch the dashboard locally:

1. Install required Python dependencies:

```
pip install streamlit pandas openpyxl matplotlib seaborn plotly
```

2. Ensure `esr_data_long.xlsx` and `form.jpg` are present in the working directory.
3. Launch the Streamlit server:

```
streamlit run app.py
```